Module
Data Collection and Analysis PGR215

Due date for submission
(see Wiseflow)

Module leader and e-mail
Noha El-Ganainy |Noha.El-Ganainy@kristiania.no

Teacher and e-mail
Arvind Keprate| arvindke@oslomet.no

Learning outcomes

After successfully completing the course the student:

Knowledge
The student:
- understands how to effectively collect data and clean it
- understands and assess how data influences algorithms, and being able to counter these influences with different methods or changes in data collection.
- can summarize the differences between CPU and GPU computing
- demonstrates a deep understanding of GPU computing in the context of AI

Skills
The student:
- can select and apply an efficient method for data collection
- can select and apply efficient methods for data preparation and cleaning
- can compare the performance of running programs on CPUs versus GPUs and profile performance

General competence
The student...
- can discuss theoretical and practical aspects of data collection
- can discuss methods and theoretical approaches for data cleaning
- can discuss theoretical aspects of parallel computing and GPU architectures

Exam specification

1. Group Size=Only 1 (Individual Submission)
2. Submit a report in PDF format and a fully executable Jupyter Notebook, with proper code and markdown cells. Remember you are graded in the basis of your PDF report and the code should be fully executed and will act as a supplement.

Please address the following questions in your submission.

Question 1: Building an ETL Pipeline (35 Marks)

You are required to build a robust ETL pipeline that extracts cryptocurrency data from two different sources, performs complex time-series transformations and string manipulation, loads the data into a database, visualizes the results using subplots, and extracts insights by merging the datasets.

Part A: Extraction (10 Marks)

1. **API Extraction - Historical Data (5 Marks):**
    - Use the CoinGecko API to fetch the 14-day historical daily market data (prices, market caps, and total volumes) for three assets: Bitcoin, Ethereum, and Solana.
    - Endpoint format: https://api.coingecko.com/api/v3/coins/{coin}/market_chart?vs_currency&days&interval
    - Extract and flatten the JSON response into a single tabular pandas DataFrame containing: coin_id, timestamp, price, and volume.

2. **Web Scraping - Complex Elements (5 Marks):**
    - Scrape the CoinMarketCap homepage (https://coinmarketcap.com/))
    - Extract data for the top 15 cryptocurrencies into a panda DataFrame. You must extract: name, symbol, price, and the 7d_change percentage.

Note: Extract the raw HTML text for the 7d_change exactly as it appears, even if it contains percentage signs or other characters.

Part B: Transformation & Feature Engineering (8 Marks)

1. **Regex / String Cleaning (3 Marks):** Clean the 7d_change column from your scraped dataset by removing the % symbol and any surrounding whitespace, then convert it to a numeric float data type. Ensure the scraped price is also cleaned (remove $ and commas) and converted to a numeric type.
2. **Time-Series Formatting (2 Marks):** Convert the Unix timestamps from the CoinGecko API data into standard human-readable YYYY-MM-DD date formats. Drop the original timestamp column.
3. **Rolling Average Calculation (3 Marks):** Group the API dataset by coin_id and calculate a 3-day rolling (moving) average for the price. Add this as a new column called price_3d_mavg. Handle any resulting NaN values by filling them with the original price for that day.

Part C: Data Loading and Verification (5 Marks)

1. **Database Creation & Insertion (3 Marks):** Create a SQLite3 database called crypto_etl.db. Store the transformed API data in a single table called historical_data, and the cleaned scraped data in a table called current_data.
2. **SQL Verification (2 Marks):** Using pandas.read_sql(), write and execute a SQL query that selects the top 5 rows from the historical_data table where the coin_id is exactly 'solana'. Print the resulting DataFrame.

Part D: Data Visualization (6 Marks) Using Matplotlib or Seaborn, create a single figure containing two subplots side-by-side (1 row, 2 columns). Subplot 1 should display Bitcoin's 14-day historical data, and Subplot 2 should display Ethereum's.

Part E: Dataset Insights (6 Marks) Using the data you successfully loaded into your SQLite database (or your merged pandas DataFrames), write the code to answer the following questions and provide brief written explanations.

1. **Data Merging & Price Discrepancy (2 Marks):** Isolate the most recent day's price for Bitcoin and Ethereum from your historical_data (API). Join this with the current_data (Scraped) for the exact same coins. Calculate and print the absolute dollar difference between the API's latest price and the scraped price for both assets.
2. **Trend Verification (2 Marks):** Your scraped data contains a 7d_change percentage. Using your historical_data, calculate your *own* 7-day percentage change for Bitcoin (comparing the most recent day's price to the price exactly 7 days prior). Print both the scraped 7d_change and your newly calculated 7-day change side-by-side.
3. **Critical Reflection (2 Marks):** In 2–3 sentences, explain why your calculated 7-day change from the API might not perfectly match the 7d_change reported on CoinMarketCap. *(Hint: Think about time zones, aggregation methods, and exact timestamps).*

Question 2: Comparative Data Analysis and Fare Prediction using NYC Yellow & Green Taxi Data (40 Marks)

For this task, you will process millions of rows of bicycle-sharing data to uncover commuter trends, seasonal changes, and system bottlenecks. You will extract large amounts of raw data, clean and engineer features using PySpark, and visualize your aggregated findings.

Dataset: NYC Citi Bike Trip Data (Public AWS S3 Bucket) https://s3.amazonaws.com/tripdata/

Part A: Programmatic Data Extraction (10 Marks)

You are required to programmatically download and extract a large batch of data. Do not download these files manually via your browser.

1. Write a Python script using the requests and zipfile (or urllib) libraries to download the monthly trip data ZIP files for the **first 6 months of 2024** (January through June). **(4 Marks)**

Note: File naming format example: 202401-citibike-tripdata.zip

2. Extract the contents of these ZIP files into a local folder named citibike_data/.**(3 Marks)**
3. Write code to list the extracted CSV files and print the total size (in MB or GB) of the extracted data directory. **(3 Marks)**

Part B: Big Data Cleaning and Feature Engineering with PySpark (15 Marks)

1. **Load Data:** Initialize a Spark session and load all 6 extracted CSV files into a single PySpark DataFrame called bike_df. **(3 Marks)**
2. **Schema & Nulls:** Print the schema. Drop any rows that contain null values in the start station or end station columns. **(3 Marks)**
3. **Feature Engineering:**
   - Convert the started_at and ended_at string columns to proper PySpark Timestamp types. **(2 Marks)**
   - Create a new column called trip_duration_mins by calculating the difference in minutes between ended_at and started_at. **(3 Marks)**
   - Extract the day_of_week (e.g., Monday, Tuesday) and the hour_of_day (0-23) from the started_at timestamp into two new columns. **(2 Marks)**

4. **Data Filtering:** Filter out anomalous data. Keep only trips where trip_duration_mins is strictly greater than 1 minute and less than 1440 minutes (24 hours). **(2 Marks)**

Part C: Exploratory Data Analysis & Aggregation (10 Marks)
Using PySpark DataFrame operations or Spark SQL, perform the following aggregations. For each result, convert the final aggregated output to a panda DataFrame and display it in your notebook.

1. **User Behavior:** Calculate the average trip_duration_mins and total trip count grouped by member_casual (subscriber vs. casual rider). **(3 Marks)**
2. **Top Routes:** Find the top 5 most frequent routes. A "route" is defined as a unique combination of start_station_name and end_station_name. **(4 Marks)**
3. **Save Aggregation:** Save the "User Behavior" aggregated pandas DataFrame to a SQLite database named citibike_summary.db as a backup. **(3 Marks)**

Part D: Advanced Data Visualization (5 Marks)
Using Matplotlib or Seaborn, create the following visualizations based on your cleaned data. *(*Hint: You should aggregate the data in PySpark first, then convert just the small aggregated result to pandas for plotting to avoid memory crashes*).*

1. **Time Distribution (Heatmap):** Create a heatmap showing the total number of trips by day_of_week (Y-axis) and hour_of_day (X-axis). Ensure the days of the week are ordered correctly (Monday to Sunday). **(3 Marks)**
2. **Monthly Trend (Line Chart):** Extract the month from the started_at column. Plot a line chart showing the total number of trips per month (Jan through Jun), with two separate lines distinguishing "member" trips vs. "casual" trips. **(2 Marks)**

Note: Both charts must have clear titles, axis labels, and legends where appropriate.

Question 3: Build a Global Earthquake Tracker App Using Streamlit (25 Marks)

Your task is to build an interactive Streamlit application that allows users to explore recent global seismic activity using the public USGS Earthquakes API.

API Endpoint (Past 30 Days of Earthquakes):
https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_month.geojson

Part A: Data Extraction and Preprocessing (5 Marks)
Before building the app, you must write a script to fetch and prepare the data.

1. **Extract:** Fetch the JSON data from the provided USGS endpoint. Extract the following fields from the features array into a pandas DataFrame (named quake_df):
    - place (from properties)
    - mag (magnitude, from properties)
    - time (from properties)
    - longitude (from geometry coordinates)
    - latitude (from geometry coordinates)
    - depth (from geometry coordinates)
2. **Clean & Transform:**
    - The time field is in Unix milliseconds. Convert this column to a standard pandas Datetime format.
    - Drop any rows that have missing values in the mag (magnitude) or coordinates columns.
    - Create a new column called risk_category based on magnitude:
        - Mag < 3.0: "Minor"
        - Mag 3.0 to 4.9: "Moderate"
        - Mag >= 5.0: "Strong"

3. **Report:** In your Jupyter Notebook, print the first 20 rows and shape your final quake_df.

Part B: Streamlit App Development (20 Marks)
Using your data extraction script as a base, build a functional, single-page Streamlit application (app.py) that includes the following features.

Requirement	Description	Marks
App Layout & Structure	Set a descriptive page title, organize the layout neatly, and add a sidebar for user navigation/controls.	3
Magnitude & Depth Filters	Include a slider in the sidebar to filter earthquakes by a minimum mag (magnitude), and another to filter by maximum depth.	3
Risk Category Filter	Include a multiselect or dropdown widget allowing users to filter by the risk_category ("Minor", "Moderate", "Strong").	3
Data Summary Metrics	Use st.metric() or similar cards at the top of the app to display the Total number of earthquakes and the Maximum magnitude currently filtered.	3
Geospatial Visualization	Plot the filtered earthquake locations on a map.	3
App Description	Write a brief markdown paragraph at the top of the app describing what the dashboard does and how to use the filters.	5

Note: Remember to submit app.py file and also include the screenshot of the running app in your report.

Assignment criteria*

Grade	Learning Outcome 1: Knowledge	Learning Outcome 2: Skills	Learning Outcome 3: Competence
A Excellent	Excellent and comprehensive understanding of concepts	Demonstrates excellent analytical, technical and writing skills	Outstanding degree of judgment and independent critical thinking
B Very good	Very good understanding of concepts	Demonstrates very good analytical, technical and writing skills	Sound degree of judgment and independent critical thinking
C Good	Good understanding of theory in most important areas	Demonstrates good analytical, technical and writing skills	Reasonable degree of judgment and independent critical thinking
D Satisfactory	Satisfactory understanding of theory, but with significant shortcomings	Demonstrates limited analytical, technical and writing skills	Limited degree of judgment and independent critical thinking
E Sufficient	Meets the minimum understanding of concepts	Demonstrates sufficient analytical, technical and writing skills	Very limited degree of judgment and independent critical thinking
F Fail	Fail to meet the minimum academic criteria.	No demonstration of analytical, technical and writing skills	Absence of judgment and independent critical thinking

*Adapted from The Norwegian Association of Higher Education Institutions